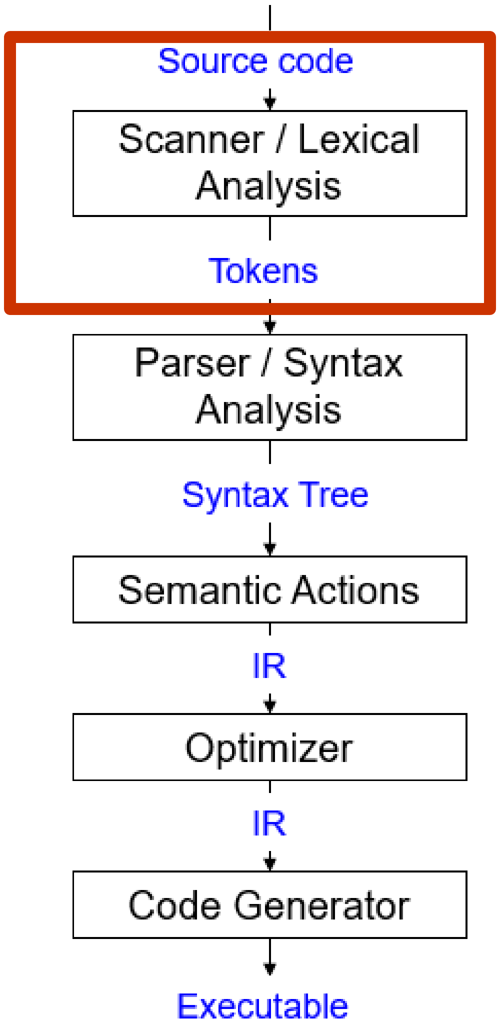
CS620T: Compilers - Principles and Implementation

Autumn 2026

Week 2: Design and Implementation of scanners and parsers

Scanners (Summary)

- Also called *Lexers* / *Lexical Analyzers*
- Input: stream of letters (program text / source code), Output: sequence / list of *tokens*
- Token: a pair <category/class, value>
  - Category defines a string *pattern*
  - Value also called *lexeme*
  - Value is a *prefix* (and hence, is a substring)
  - Value matches on of the patterns that category defines
- Scan *left-to-right* in program text, *look-ahead* to identify tokens.
  - Look-ahead buffer size determined by language design



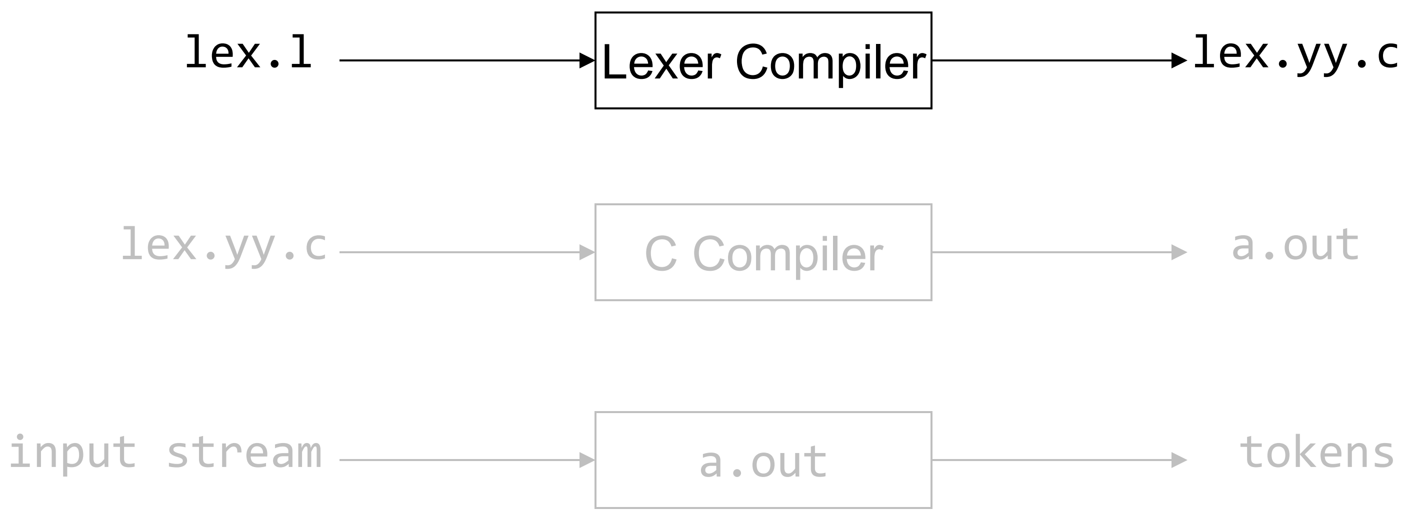
Summary

- We saw what it takes to write a scanner:
  - Specify how to identify token classes (using regexps)
  - Convert the regexps to code that identifies a *prefix* of the input program text as a *lexeme* matching one of the token classes
    - Use tools for automatic code generation (e.g. `Flex / ANTLR`)
      - *How do the tools convert regexps to code? Finite Automata*

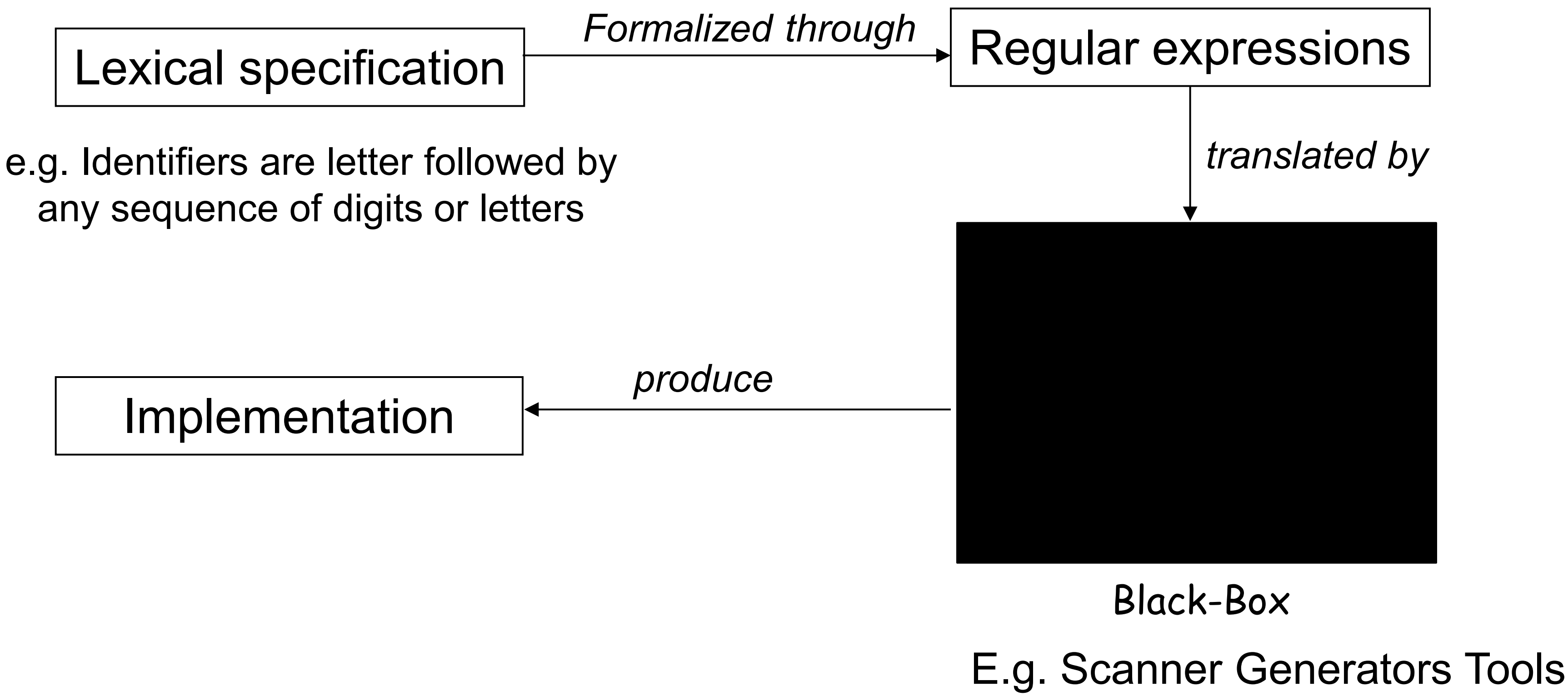
OR

    - Scanner code manually

Summary: Lex (Flex)



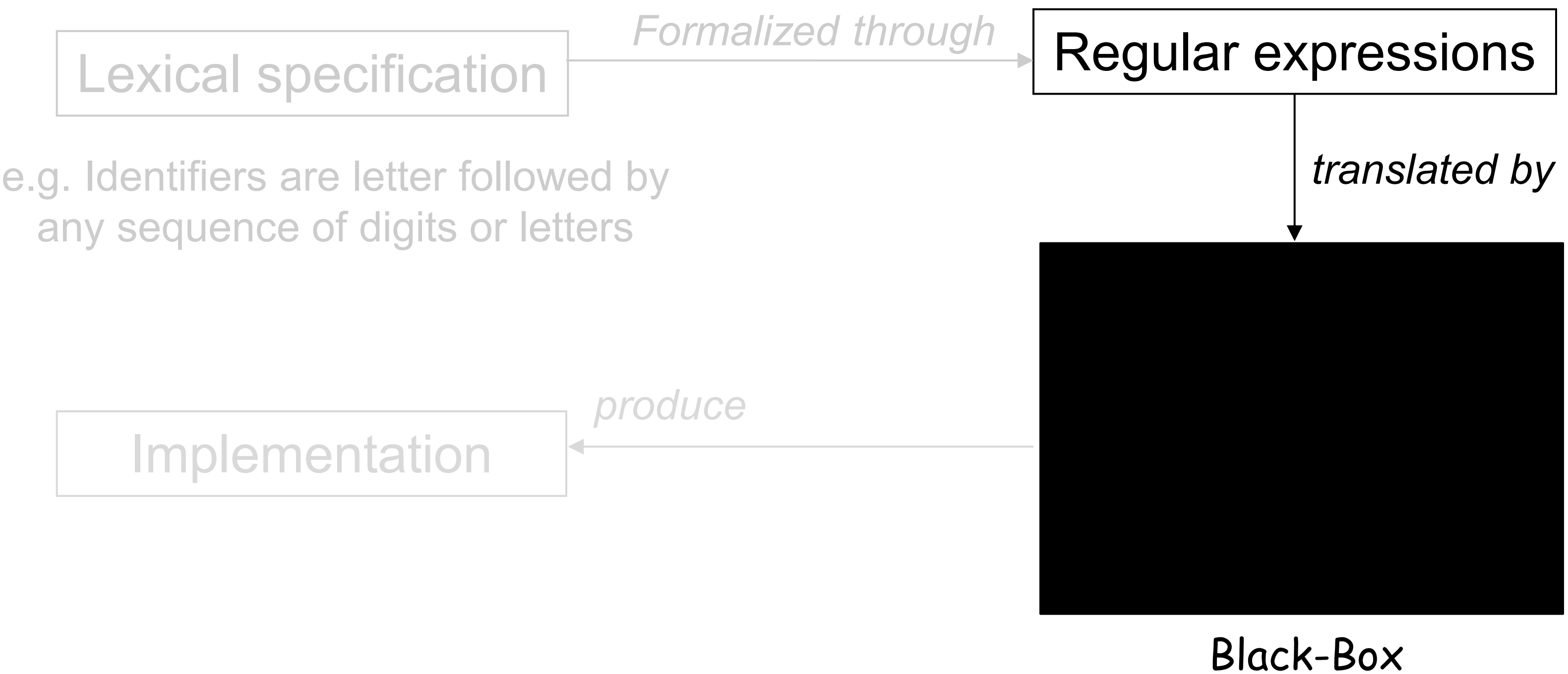
Summary: Scanner / Lexical Analyzer - flowchart



Summary: Regular Expressions for Lexical Specifications

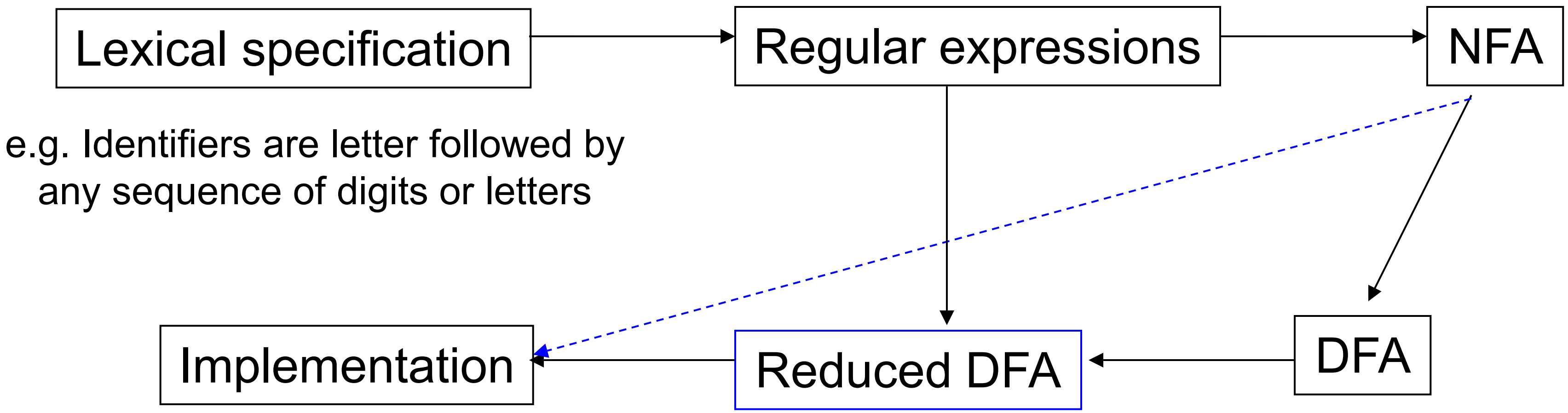
- Digit: D = (0|1|2|3|4|5|6|7|8|9)
  alternative definition: [0-9]
- Letter: L = [A-Za-z]
- Literals (integers or floats): -?D+(.D*)?
- Identifiers: (_|L)(_|L|D)*
- Comments (as in Micro): //Not(\n)*\n
- More complex comments (delimited by ##, can use # inside comment):
  ## ((#|λ) Not(#))* ##

Scanner- Implementation



How does a tool such as Flex generate code?

Scanner - flowchart



Discussion

- Why separate class (token type) for each keyword?
  - Efficiency
    - Parsers take decisions based on token types. When decision making not possible, switch to token values, which are strings. String comparison is more expensive
  - Compatibility with parser generators
    - Some parser generators don’t support semantic predicates
  - Autocomplete / Intellisense

Discussion - Efficiency

```
switch(curToken.type) {
      case IF: parse_if_stmt();
               break;
      ..
}
```

```
switch(curToken.type) {
      case KEYWORD: if(curToken.value==“if”);
               parse_if_stmt();
      ..
}
```

Discussion - Compatibility

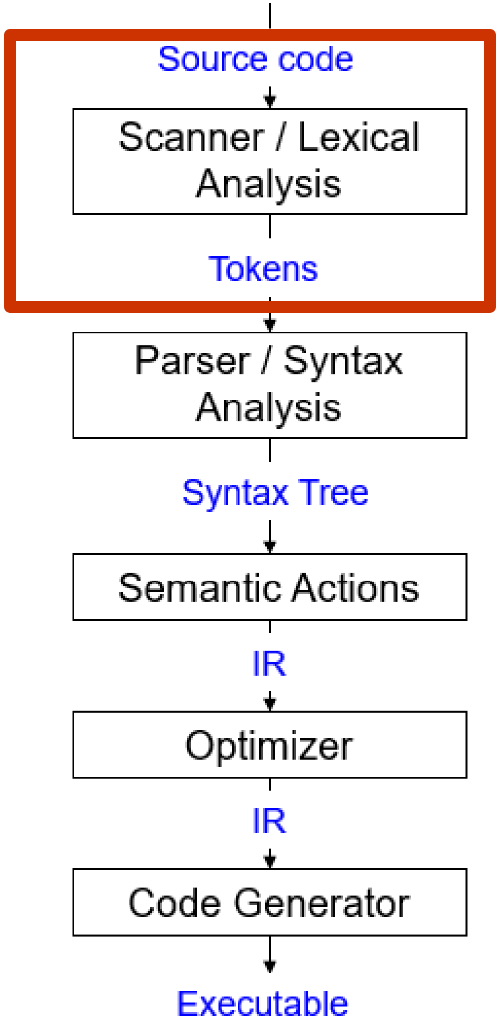
```
statement : IF condition body (ELSE body)? FI

statement : if condition body (else body)? fi
if: {current_token.value == "if"} KEYWORD ;
else: {current_token.value == "else"} KEYWORD ;
fi: …
KEYWORD: IF | ELSE | FI
```

Suggested Reading

- Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D.Ullman: Compilers: Principles, Techniques, and Tools, 2/E, AddisonWesley 2007
  - Chapter 3 (Sections: 3.1, 3,3, 3.6 to 3.9)
- Fisher and LeBlanc: Crafting a Compiler with C
  - Chapter 3 (Sections 3.1 to 3.4, 3.6, 3.7)

Scanners (Summary)

- *Regular expressions* are used to formally define the patterns specified by token classes.
  - Some customization done while defining regular expressions: 1) Match the longest substring possible 2) Handle errors
- Tools such as Flex and ANTLR convert regular expressions to code. The code is your scanner implementation
  - The implementation typically converts regular expressions to *Finite Automata* (special kind of state diagram)
    - Automata are coded using efficient algorithms (E.g. Table-lookup method)
  - Efficient algorithms exist for substring matching (requiring single-pass over input program text)
    - Aho-Corasic, Knuth-Morris-Pratt (KMP)



Parsers - Overview

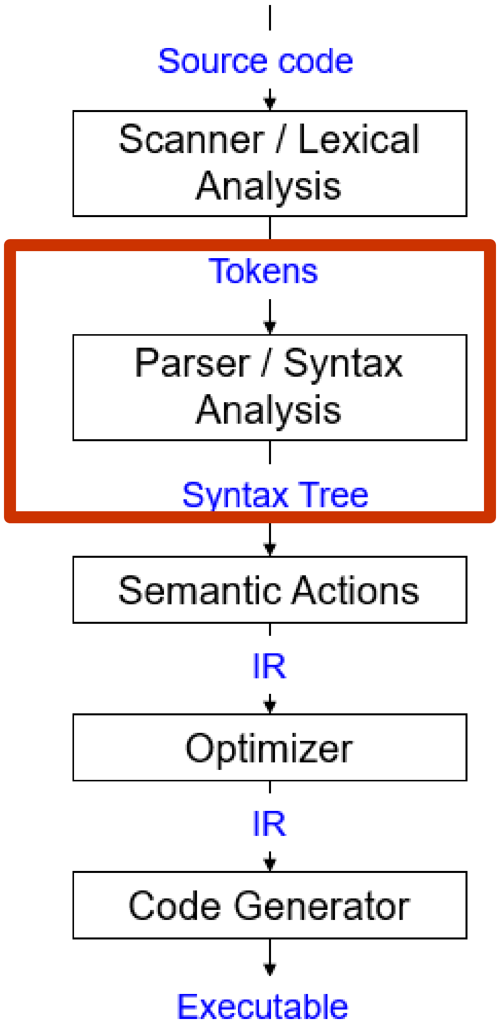
- Also called syntax analyzers
- Determine two things:
  1. Is a program syntactically valid?

     (Analogy) is an English language sentence grammatically correct?
  2. What is the structure of programming language constructs? E.g. does the sequence*

```
IF, ID(a), OP(<), ID(b), {, ID(a),
ASSIGN, LIT(5), }}
```

     refer to an `if` statement?

     (Analogy) diagramming English sentences



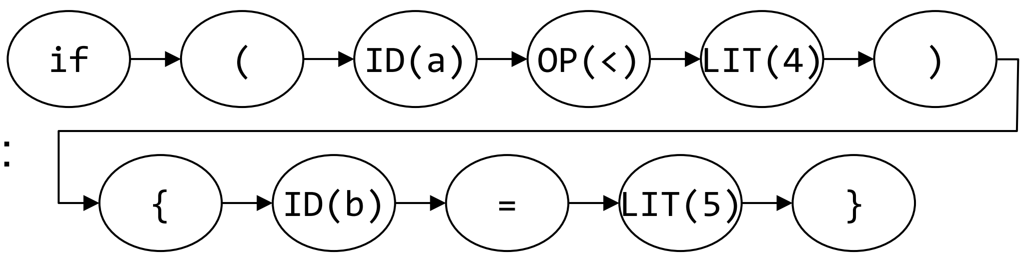
* Correpending program text:

```
if (a < 4) {
     b = 5
}
```

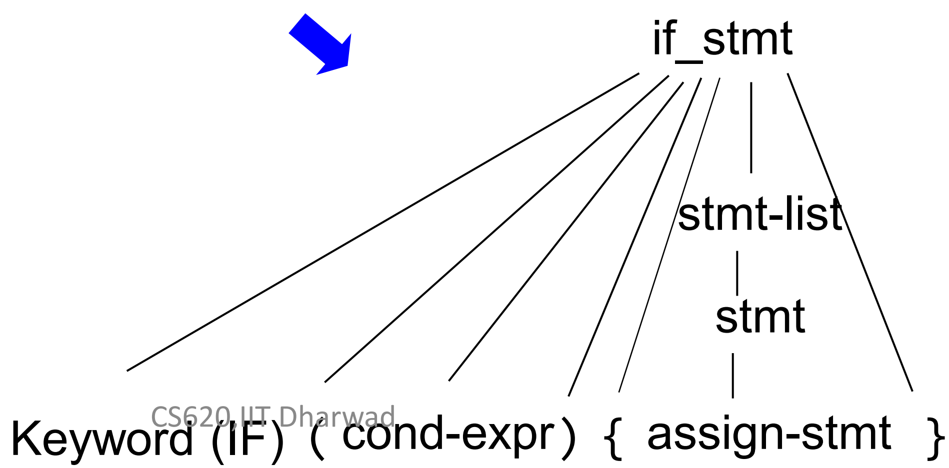
Parsers - Overview

- Input: stream of tokens
- Output: Parse tree
  - sometimes implicit

Stream of tokens:



Parse tree:



Parsers – what do we need to know?

1. How do we define language constructs?
   - Context-free grammars
2. How do we determine: 1) valid strings in the language? 2) structure of program?
   - LL Parsers, LR Parsers
3. How do we write Parsers?
   - E.g. use a parser generator tool such as Bison

Pre-Exercise

Which of the below strings are accepted by the grammar:

```
1: A ->  aAa
2: A ->  bBb
3: A ->  λ
4: B ->  cA
5: B ->  λ
```

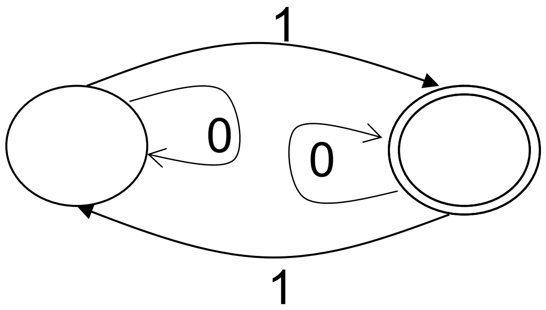
1. abcba   1->2->4->3
2. abcbca
3. abba   1->2->5
4. abca

Languages

- A language is (possibly infinite) set of strings
- Regular expressions specify *regular languages.* However, regular languages are *weak formal languages* to describe the features of a practical programming language.

What set of strings does this FA accept?

The FA shown accepts all string with *odd number of 1s.*



What is the regular expression for the FA?

(0*10*)(10*10*)*

Regular expressions can describe strings specifying *parity*:

{ mod k | k=# states in FA}

weakness: regular expressions can't describe a string of the form: $\{ (^i\ )^i \mid i>=1\}$

Regular Languages

- Regular expressions can't describe a string of the form:

$$\{\ (^{i}\ )^{i}\ \mid\ i>=1\}$$

E.g. Parenthesized expressions

```
((2+3)*5)
```

Programming language syntax is i.e. recursive

```
((( int x; )))
```

Nested structures:

```
IF
  IF
    IF
    FI
  FI
FI
```

Context Free Grammar (CFG)

- Natural notation for describing <u>recursive structure</u> definitions. Hence, suitable for specifying language constructs.
- Consist of:
  - A set of *Terminals* (T)
  - A set of *Non-terminals* (N)
  - A *Start Symbol* (S∈N)
  - A *set of Productions* ($X \to Y_1 .. Y_N$) (aka. rules)

$$P: X \longrightarrow Y_1 Y_2 Y_3 .. Y_N \qquad X \in N, \qquad Y_i \in N \cup T \cup \epsilon/\lambda$$

Context Free Grammar (CFG)

- Grammar `G = (T, N, S, P)`

  E.g. $G = (\{a,b\}, \{S, A, B\}, S, \{S \rightarrow AB, A \rightarrow Aa$
  $A \rightarrow a, B \rightarrow Bb, B \rightarrow b\})$

- Implicit meanings
  - <u>First rule</u> listed in the set of productions contains <u>start symbol</u> (on the left-hand side)
  - In the set of productions, <u>you can replace the symbol X</u> (appearing on the right-hand side only) with the <u>string of symbols</u> that are on the right-hand side of a rule, which has X (on the left-hand side)

Context Free Grammar (CFG)

1. Begin with only S as the initial string
2. Replace S
   - S replaced with AB
3. Repeat 2 until the string contains only terminals
   i. AB replaced with aB
   ii. aB replaced with ab

```
G = (T, N, S, P)
P:{ S->AB,
  A->Aa,
  A->a,
  B->Bb,
  B->b }
```

Summary: we move from S to a string of terminals through a series of transformations:

α_0-> ... -> α_n where $\alpha_1 \ldots \alpha_n$ are strings

Shorthand notation: $\alpha_0 \overset{*}{\rightarrow} \alpha_n$

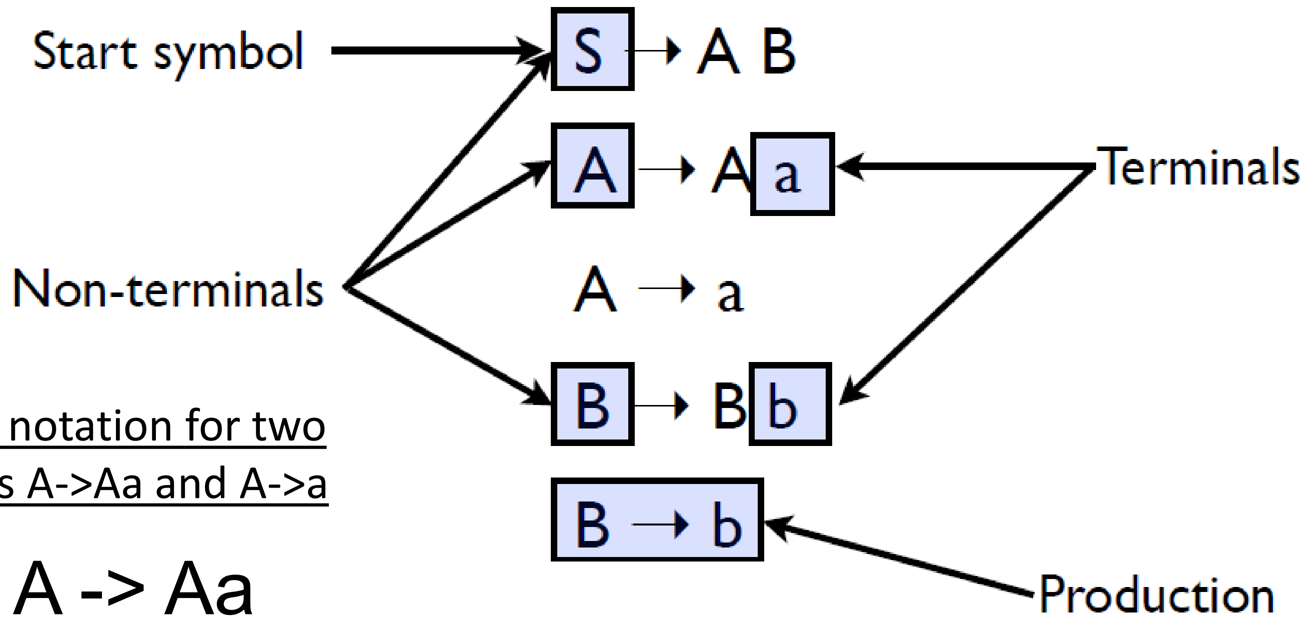
Detour: Context-Sensitive Grammar

- Can have context-sensitive grammar and languages (think: aB->ab)
  - Cannot replace right-hand side with left-hand side irrespective of the context.
  - E.g. aB->ab lays down a context: ‘a’ must be a prefix in order to transform the string “aB” to a string of terminals “ab”
    - ccaBb can be replaced by ccabb

Is grammar G context-free?

```
G = (T, N, S, P)
P:{ S->AB,
   A->Aa,
   A->a,
   B->Bb,
   B->b }
```

Simple grammar (Summary)



<u>Alternative notation for two productions A->Aa and A->a</u>

A -> Aa
| a

Backus Naur Form (BNF)

Programming language syntax

- Programming language syntax is defined with CFGs
- Constructs in language become non-terminals
  - May use auxiliary non-terminals to make it easier to define constructs

if_stmt → if (cond_expr) then statement else_part

else_part → else statement

else_part → λ

- Tokens in language become terminals

Language of the Grammar

- Language L(G) of the context-free grammar G
  - Set of strings that can be derived from S
  - $\{a_1a_2a_3..a_N \mid a_i \in T\ \forall i \text{ and } S \overset{*}{\rightarrow} a_1a_2a_3..a_N\}$
  - Is called context-free language
    - All regular languages are context-free but not vice-versa.
    - Can have many grammars generating same language.

String Derivations: Does a string belong to the Language?

- How do we apply the grammar rules to determine the acceptability of a string? (i.e. the string belongs to the language, *L(G)*, specified by the CFG *G*)
  - Begin with S
  - Replace S
  - Repeat till string contains terminals only. Why terminals only?

L(G) must contain strings of terminals only

- Notation:
  - We will use Greek letters to denote strings containing non-terminals and terminals
- Derivations: sequence of rules applied to produce the string of terminals

Generating strings (Example)

$S \rightarrow A\ B$

$A \rightarrow A\ a$

$A \rightarrow a$

$B \rightarrow B\ b$

$B \rightarrow b$

- Given a start rule, productions tell us how to rewrite a non-terminal into a different set of symbols
- Some productions may rewrite to λ. That just removes the non-terminal

To derive the string "a a b b b" we can do the following rewrites:

$$S \Rightarrow A\ B \Rightarrow A\ a\ B \Rightarrow a\ a\ B \Rightarrow a\ a\ B\ b \Rightarrow$$
$$a\ a\ B\ b\ b \Rightarrow a\ a\ b\ b\ b$$

CFG and Parsers

- Is it enough if parsers answer “yes” or “no” to check if a string belongs to context-free language?
  - Also need a parse tree
- What if the answer is a “no”?
  - Handle errors
- How do we implement CFGs?
  - E.g. Bison

Post-Exercise

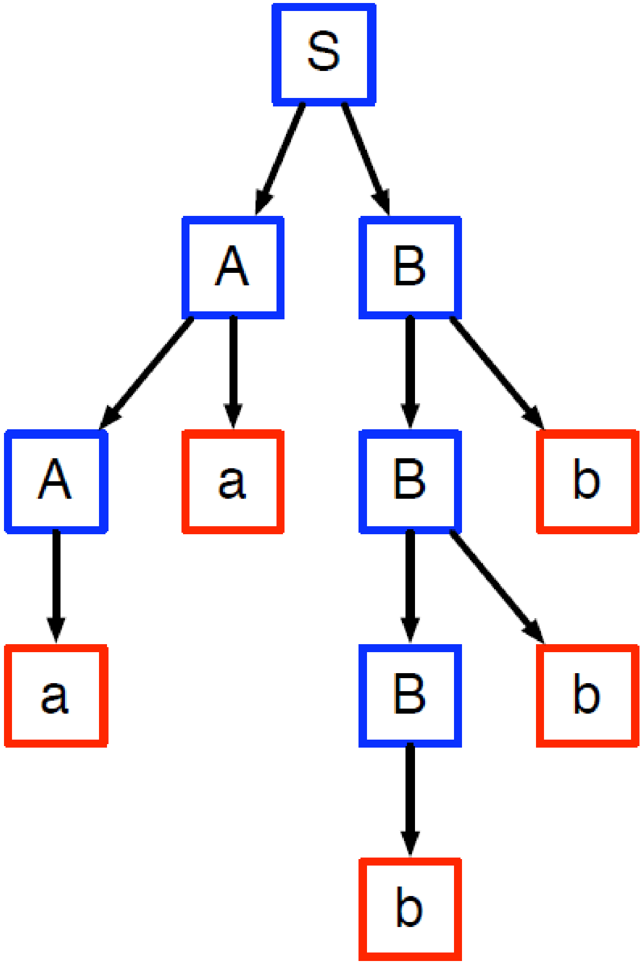
Which of the below strings are accepted by the grammar:

```
1: A ->  aAa
2: A ->  bBb
3: A ->  λ
4: B ->  cA
5: B ->  λ
```

1. abcba 1->2->4->3
2. abcbca
3. abba 1->2->5
4. abca

Parse trees

- Tree which shows how a string was produced by a language
  - Interior nodes of tree: non-terminals
    - Children: the terminals and non-terminals generated by applying a production rule
  - Leaf nodes: terminals



Derivations and Parse Trees

- Recall: Derivation is a sequence of rules applied to produce a string
  - $S \text{ -> } \alpha_0 \text{ -> } \alpha_1 \text{ -> } \alpha_2 \text{ -> } . . . \text{ ->} \alpha_n$
- A derivation defines a parse tree
  - Parse tree is an alternative way to gather information on how the string was derived
  - A parse tree may have many derivations (think: different permutations of α)

Derivations and Parse Trees

- Consider the grammar with the following rules:

```
1: E -> E + E
2:    | E * E
3:    | id
```

- Produce derivations for the string: **id*id+id**

Derivations and Parse Trees

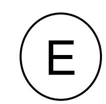
- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 1: Start with E, the start symbol **Parse Tree**

E



Derivations and Parse Trees

- Consider the grammar with the following rules:

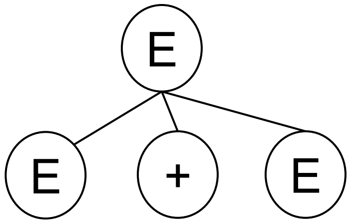
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 1: **Replace E with E + E**

E
E+E

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

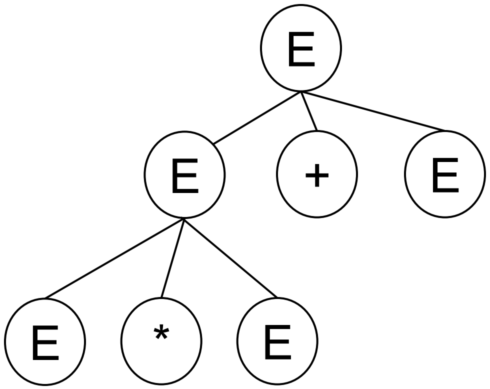
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 2: Replace E with E * E

E
E+E
E*E+E

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

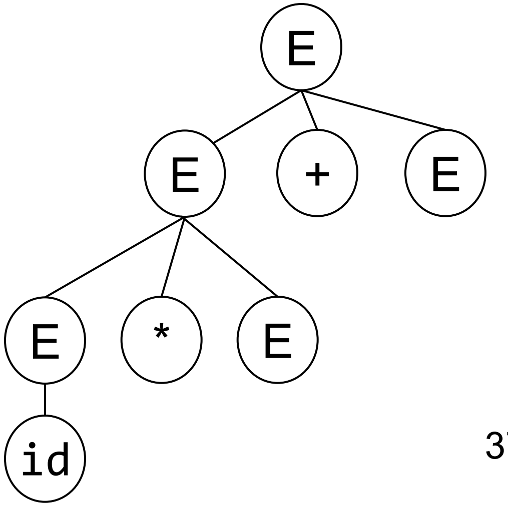
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: Replace E with id

E
E+E
E*E+E
id*E+E

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

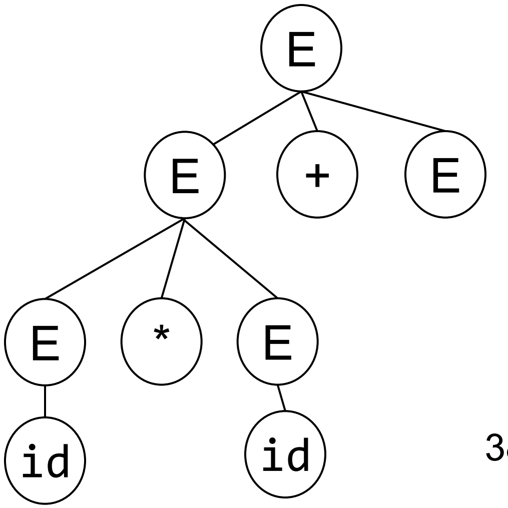
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: Replace E with id

E
E+E
E*E+E
id*E+E
id*id+E

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

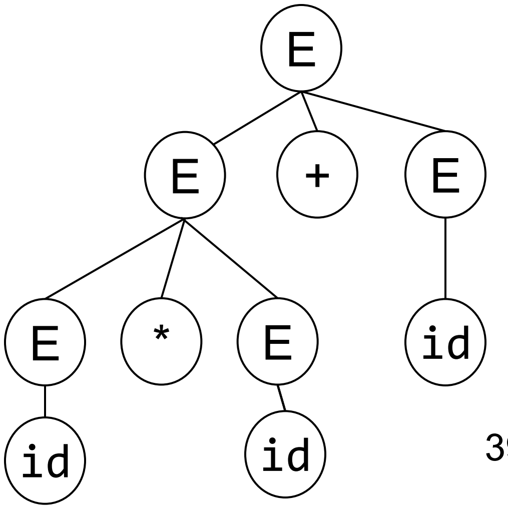
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with id**

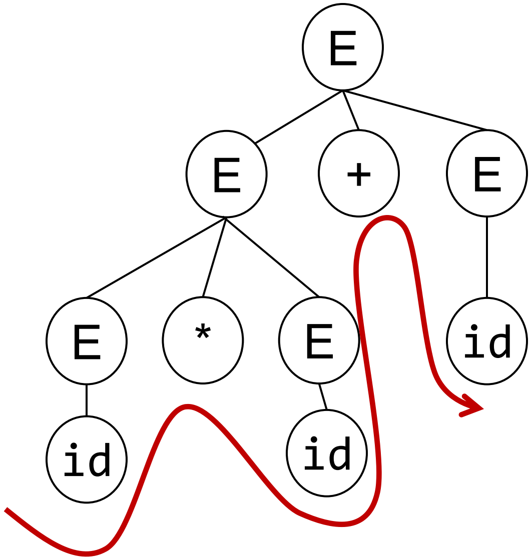
E
E+E
E*E+E
id*E+E
id*id+E
id*id+id

Parse Tree



Derivations and Parse Trees

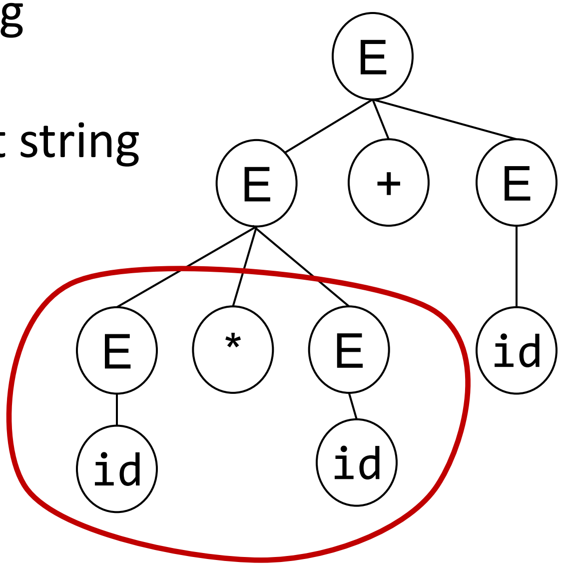
- Note in previous slides:
  - Replacement done on left-most non-terminal in the string - **called left-most derivation**
  - Terminals at leaves and non-terminal as interior nodes
  - `Inorder` traversal of leaves produces input string `id*id+id`



Derivations and Parse Trees

- Note in previous slides:
  - Replacement done on left-most non-terminal in the string - **called left-most derivation**
  - Terminals at leaves and non-terminal as interior nodes
  - `Inorder` traversal of leaves produces input string `id*id+id`
  - Parse tree shows association of operations. Input string doesn't
    - `*` associated with identifiers in the subtree

      `(id * id)+id`



Derivations and Parse Trees

- Consider the same grammar (having the following rules):

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **`id*id+id`**
  - Using **right-most derivations**

    i.e. replace the right-most non-terminal

Derivations and Parse Trees

- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Start with E, the start symbol

E

E

Parse Tree

Derivations and Parse Trees

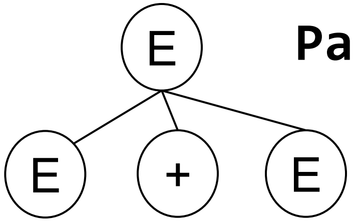
- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 2: Replace E with E+E

E
E+E



Parse Tree

Derivations and Parse Trees

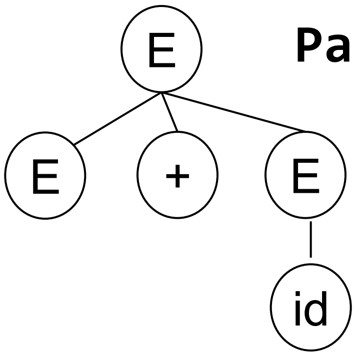
- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 1: **Replace E with id**

E
E+E
E+id



Parse Tree

Derivations and Parse Trees

- Consider the grammar with the following rules:

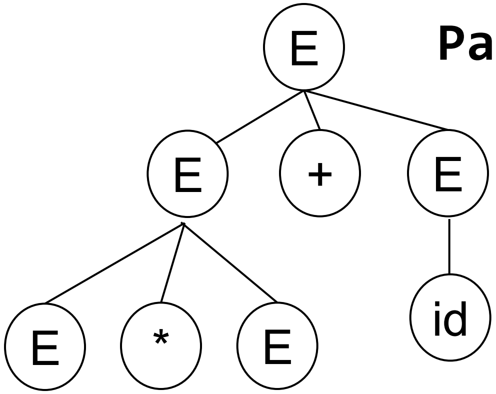
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with E * E**

E
E+E
E+id
E*E+id

Parse Tree



Derivations and Parse Trees

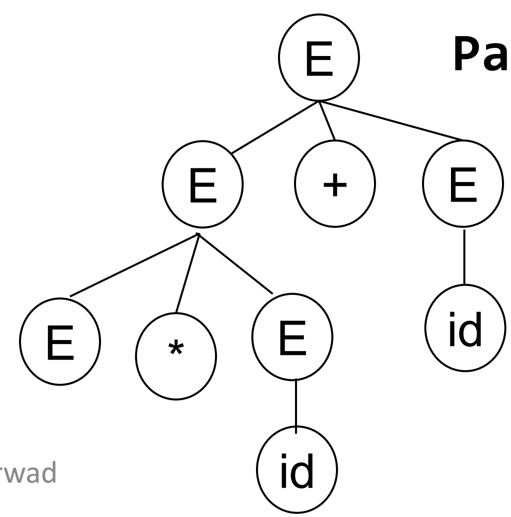
- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with id**

E
E+E
E+id
E*E+id
E*id+id



Parse Tree

Derivations and Parse Trees

- Consider the grammar with the following rules:

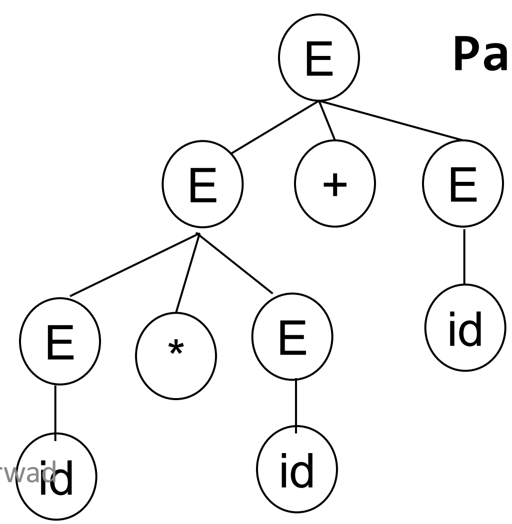
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: Replace E with id

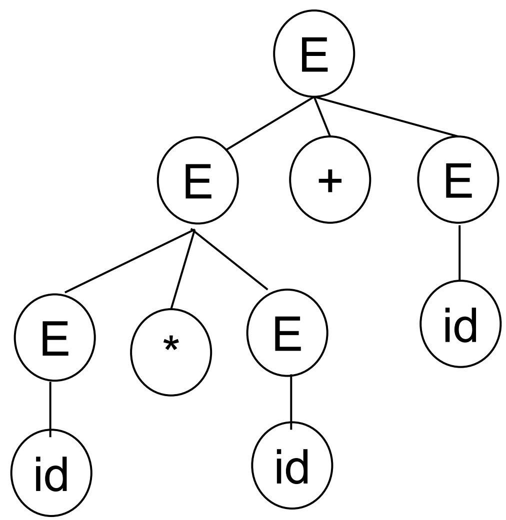
```
E
E+E
E+id
E*E+id
E*id+id
id*id+id
```

Parse Tree



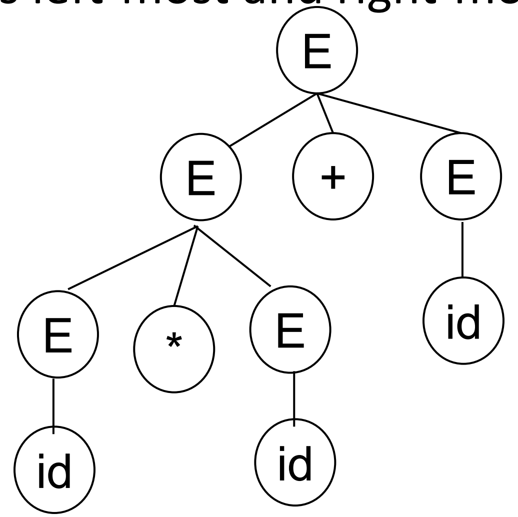
Derivations and Parse Trees

- We get the same parse tree using left-most and right-most derivations.
  - Every parse tree has left-most and right-most (and any random order) derivations.



Derivations and Parse Trees

- We get the same parse tree using left-most and right-most derivations.
  - Every parse tree has left-most and right-most (and any random order) derivations.



- But there could be a string (or more than one strings) for which there exists derivations that would get different parse trees

Derivations and Parse Trees

- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Start with E, the start symbol

E

E **Parse Tree**

Derivations and Parse Trees

- Consider the grammar with the following rules:

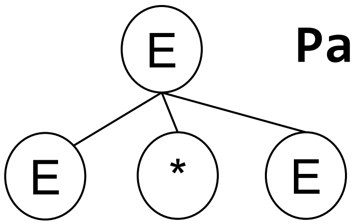
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 2: Replace E with E*E

Earlier it was replace E with E+E

E
E*E



Parse Tree

Derivations and Parse Trees

- Consider the grammar with the following rules:

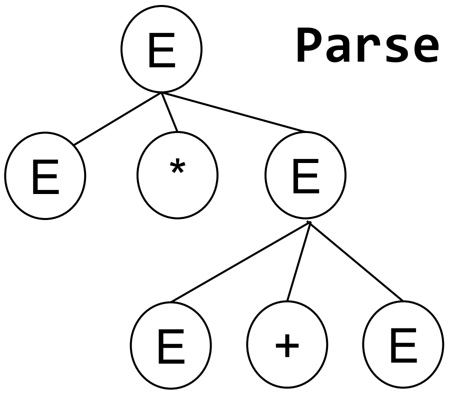
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 1: **Replace E with E+E**

E
E*E
E*E+E

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

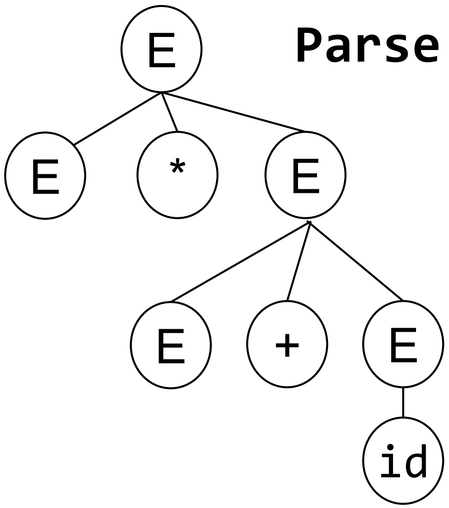
```
1: E -> E + E
2:    | E * E
3:    | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with id**

E
E*E
E*E+E
E*E+id

Parse Tree



Derivations and Parse Trees

- Consider the grammar with the following rules:

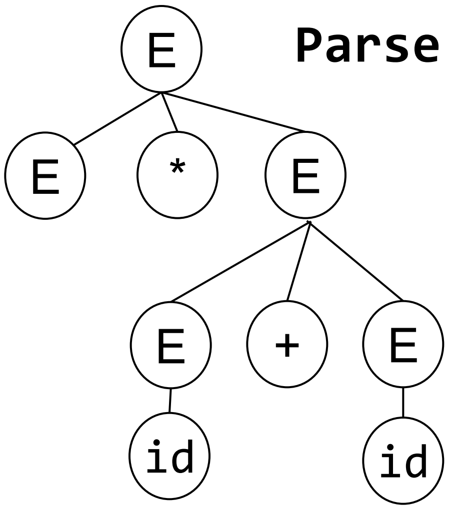
```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with id**

E
E*E
E*E+E
E*E+id
E*id+id

Parse Tree



Derivations and Parse Trees

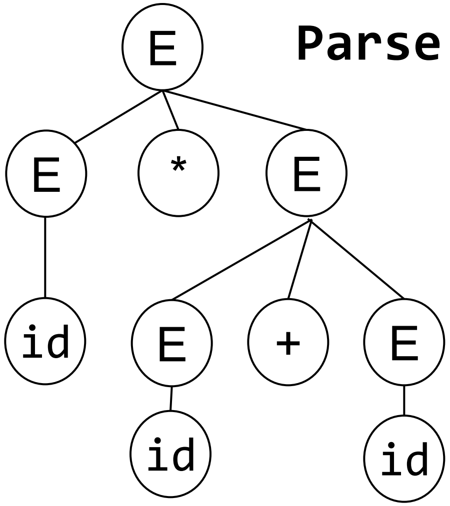
- Consider the grammar with the following rules:

```
1: E -> E + E
2:      | E * E
3:      | id
```

- Produce derivations for the string: **id*id+id**

Apply 3: **Replace E with id**

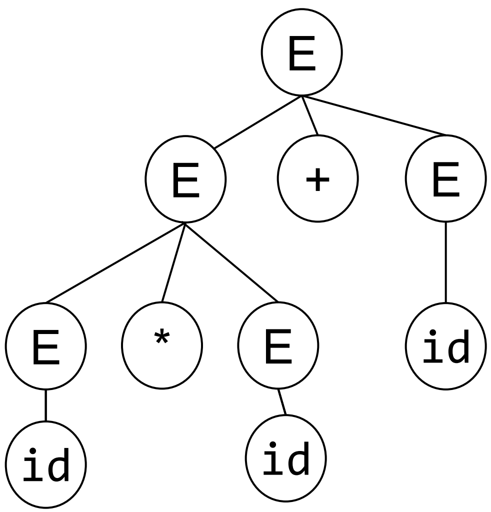
```
E
E*E
E*E+E
E*E+id
E*id+id
id*id+id
```



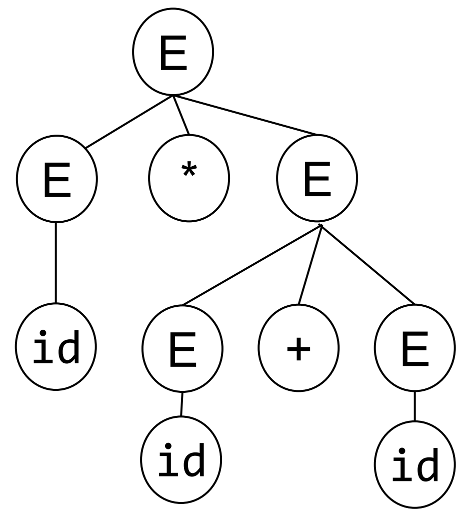
Parse Tree

Derivations and Parse Trees

- Input string: `id*id+id`



earlier



now

- `Inorder` traversal of leaves in both trees produces the same input string

Ambiguous Grammar

- Grammar that produces more than one parse tree for <u>some</u> string

```
1: E -> E + E
2:      | E * E
3:      | id
```

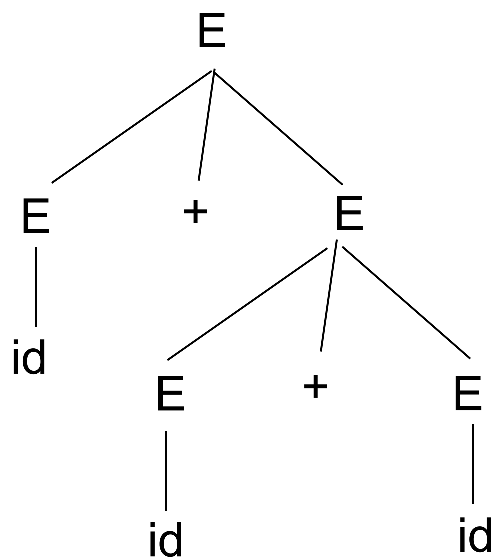
Ambiguity – what to do?

- Ignore it (let it be ambiguous)
  - Give hints to other components of the compiler on how to resolve it
- Fix it (Manually)
  - May make the grammar complicated and difficult to maintain

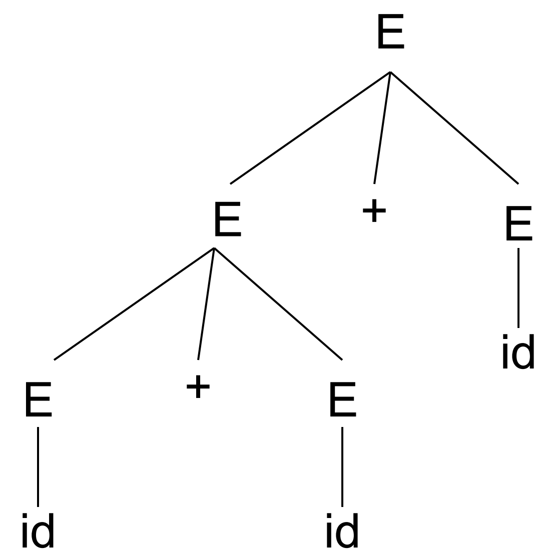
Ambiguity – ignore

- Grammar: `E -> E + E | id`   input:`id+id+id`

```
E->E+E
E->id+E
E->id+E+E
E->id+id+E
E->id+id+id
```




```
E->E+E
E->E+E+E
E->id+E+E
E->id+id+E
E->id+id+id
```



Matches the input, which would be evaluated (later) as: `id+(id+id)`

`(id+id)+id`

`%left +` **Provide hint (in Bison). Associativity declaration.**

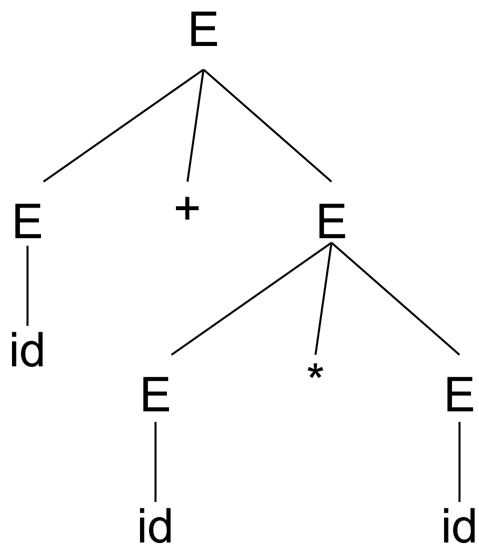
(left associative for +. So, produces the parse tree on the right)

Ambiguity - ignore

- `E -> E + E | E * E | id`

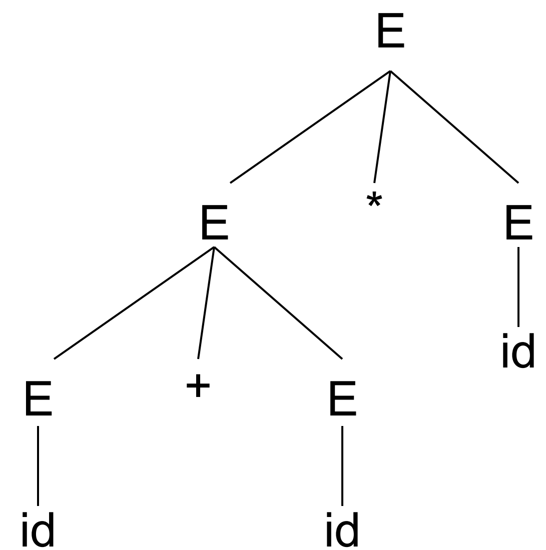
E->E+E
E->id+E
E->id+E*E
E->id+id*E
E->id+id*id

Produces tree for:
id+(id*id)



E->E*E
E->E+E*E
E->id+E*E
E->id+id*E
E->id+id*id

Produces tree for:
(id+id)*id



%left +
%left *

*Tells that * has higher precedence over + and both are left associative. So, we get the tree on left.*

Ambiguity – fixing

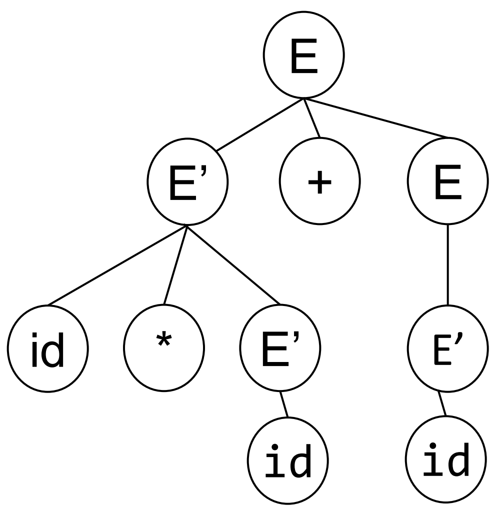
- Rewrite `E -> E + E | E * E | id` as:

```
E  -> E' + E | E'
E' -> id * E' | id
   | (E) * E' | (E)
```

If you want to handle parenthesized expressions such as (id+id)*id

Parse tree for input id*id+id

```
E->E'+E
E'->id*E'
E'->id
E->E'
E'->id
```



- E controls generation of +
- E' controls generation of *

*'s are always nested deeper in the parse tree.

is the above sequence left-most or right-most derivation?

CFG and Parsers

- Is it enough if parsers answer “yes” or “no” to check if a string belongs to context-free language?
  - Also need a parse tree
- What if the answer is a “no”?
  - Handle errors
- How do we implement CFGs?
  - E.g. Bison

Next

Error Handling

- Objective: detect invalid programs and provide meaningful feedback to programmer
  - Report errors accurately
  - Recover from errors quickly
  - Don’t slow down compilation

Error Types

- Many types of errors:
  - Lexical – `int 9abc;` //invalid identifier
  - Syntactic – extra brace inserted {
  - Semantic – `float sqr; sqr(2);`
    //use variable name with function call syntax
  - Logical – use = instead of ==

Error Handling - Types

1. Panic mode
2. Error production
3. Automatic local or global correction

Panic Mode Error Handling

- Simplest, most popular
- Discards tokens until one from a set of *synchronizing tokens* is found
- Synchronizing tokens have a clear role

  e.g. semicolons, braces
- E.g. i= i++j

  policy: while parsing an expression, discard all tokens until an identifier is found. *This policy skips the additional +*
- Specifying policy in bison: `error` keyword

```
E -> E + E | (E) | id | error id | error
```

Error Productions

- Anticipate common errors
  - 2 x instead of 2 *
- Augment the grammar
  - E -> EE | …
- Disadvantages:
  - Complicates the grammar

Error Corrections

- Rewrite the program – find a “nearby” correct program
  - Local corrections – insert a semicolon, replace a comma with semicolon etc.
  - Global corrections – modify the parse tree with “edit distance” metric in mind
- Disadvantages?
  - Implementation difficulty
  - Slows down compilation
  - Not sure if “nearby” program is intended

Parsers – what do we need to know?

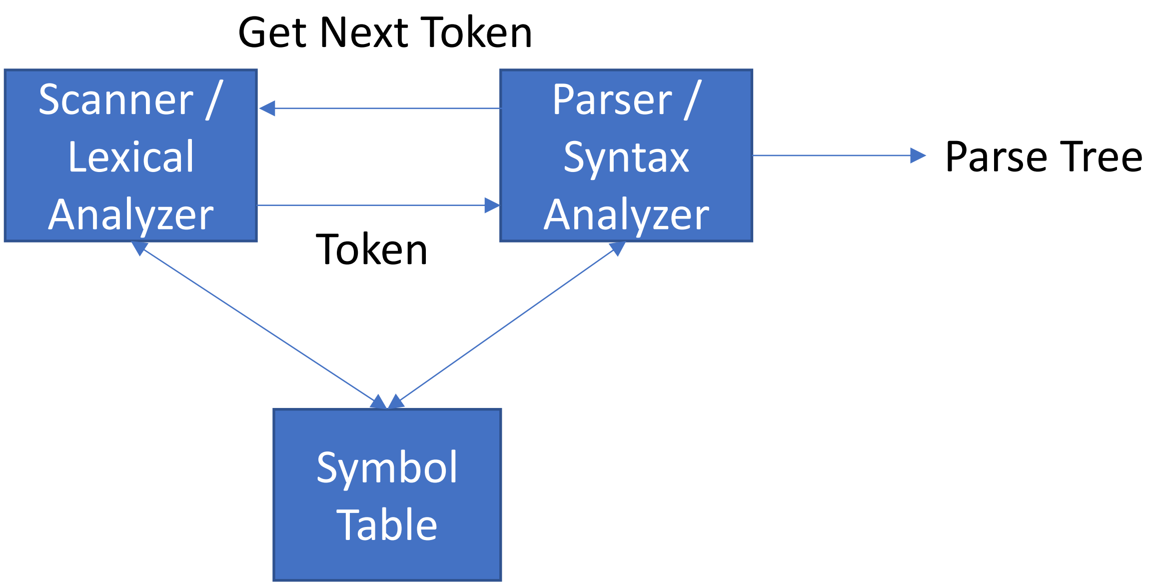
1. How do we define language constructs?
   - Context-free grammars
2. How do we determine: 1) valid strings in the language? 2) structure of program?
   - LL Parsers, LR Parsers

Next

3. How do we write Parsers?
   - E.g. use a parser generator tool such as Bison

Demo

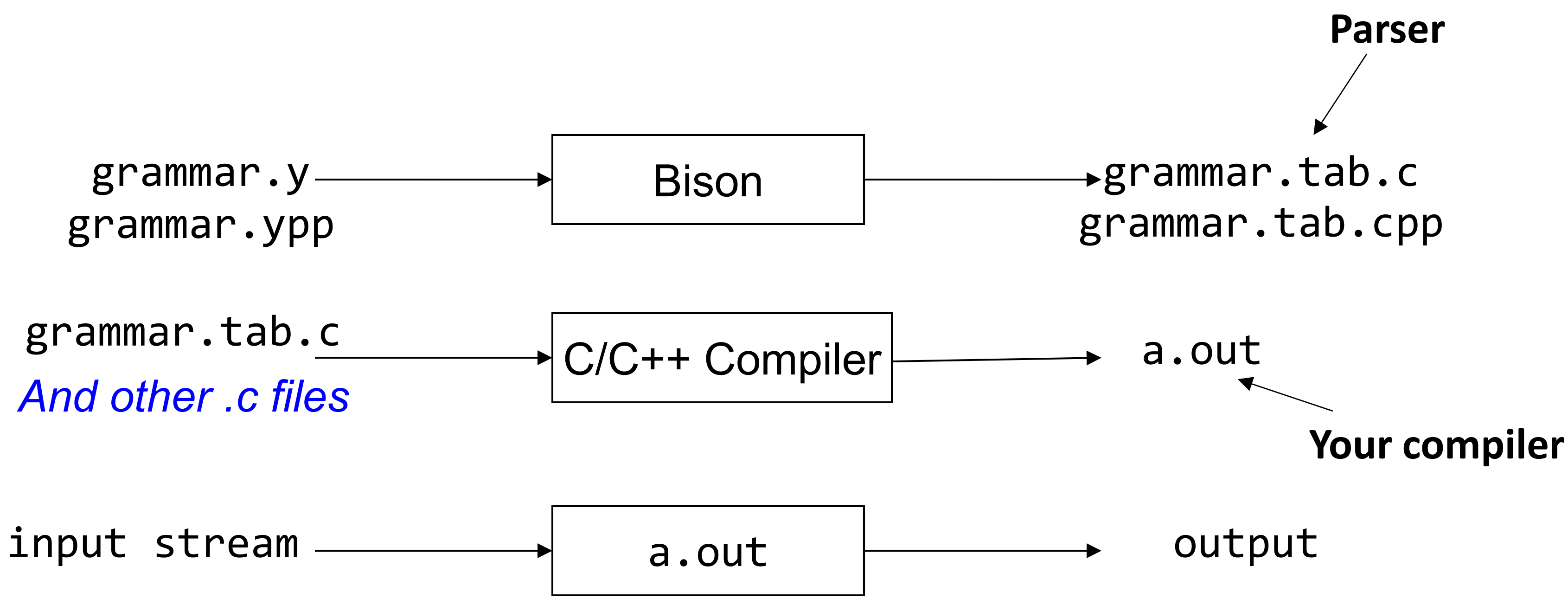
- Parser (in an implementation of a compiler)



Bison (YACC)

- Specify the grammar
- Write a lexical analyzer to process input programs and pass the tokens to parser
- Call `yyparse()` from `main`
- Write error-handlers (what happens when the compiler encounters invalid programs?)

Bison (YACC)



Parser
grammar.y
grammar.ypp
Bison
grammar.tab.c
grammar.tab.cpp
grammar.tab.c
And other .c files
C/C++ Compiler
a.out
Your compiler
input stream
a.out
output

Bison (YACC) – Input Format

```
%{
Prologue
%}
Bison declarations
%%
Grammar rules
%%
Epilogue
```

Bison (YACC) – Grammar Rules

```
%{
Prologue
%}
Bison declarations
%%
E: E PLUS E {}
 | INTEGER_LITERAL {}
 ;
%%
Epilogue
```

Bison (YACC) - Prologue

```
%{
Prologue
%}
%token PLUS INTEGER_LITERAL
%left PLUS
%%
E: E PLUS E {}
 | INTEGER_LITERAL {}
 ;
%%
Epilogue
```

Bison (YACC) - Actions

```
%{
Prologue
%}
%token PLUS INTEGER_LITERAL
%left PLUS
%%
E: E PLUS E { $$ = $1 + $3; }
 | INTEGER_LITERAL { $$ = $1; }
 ;
%%
Epilogue
```

Legal C/C++ code

Bison (YACC) – Semantic Values

```
%{
Prologue
%}
%token PLUS INTEGER_LITERAL
%left PLUS
%%        $1    $2        $3
E: E PLUS E { $$ = $1 + $3; }
 | INTEGER_LITERAL { $$ = $1; }
 ;
%%
Epilogue
```

Bison (YACC) – Helper Functions

```
%{
int yylex();
void yyerror(char *s);
%}
%token PLUS INTEGER_LITERAL
%left PLUS
%%
E: E PLUS E { $$ = $1 + $3; }
 | INTEGER_LITERAL { $$ = $1; }
 ;
%%
Epilogue
```

Bison (YACC) – Helper Functions

```
%{
#include<stdlib.h>
#include<stdio.h>
int yylex();
void yyerror(char const *s);
%}
%token PLUS INTEGER_LITERAL
%left PLUS
%%
E: E PLUS E { $$ = $1 + $3; }
 | INTEGER_LITERAL { $$ = $1; };
%%
void yyerror(char const* s) {
      fprintf(stderr,"%s\n",s);
      exit(1);
}
```

Bison (YACC) – Integrating

- Recall that terminals are tokens
- Lexer produces tokens
  - How do the parser and lexer have a common understanding of tokens?
  - How should the Lexer return tokens?

```
//grammar.y file
…
%token PLUS INTEGER_LITERAL
%%
E: E PLUS E { $$ = $1 + $3; }
 | INTEGER_LITERAL { $$ = $1; };
%%
…
```

```
bison -d grammar.y
```

grammar.tab.h

```
//scanner.l file
#include"grammar.tab.h"
extern YYSTYPE yylval
%%
\+        {return PLUS;}
[0-9]+    { yylval=atoi(yytext);
          return INTEGER_LITERAL;}
.|\n      {}
%%
…
```

- Previous slide showed how values of tokens can be passed from scanner to parser (yylval in scanner.l).
- Often, we need to pass values of different types (e.g. strings, integers, floats, etc.) to parser.

Passing data objects from lexer to parser using Flex and Bison

- Goal: recognize an INTLITERAL and STRINGLITERAL in the program text, print the line number and the value of the INTLITERAL and STRINGLITERAL as a semantic action in the parser.

```
STRINGLITERAL val:"Hello" at line:6
INTLITERAL val:1 at line:7
INTLITERAL val:2 at line:7
INTLITERAL val:0 at line:9
INTLITERAL val:10 at line:10
INTLITERAL val:1 at line:12
Accepted
```

```
1 PROGRAM test
2 BEGIN
3         INT a,b,c,x,y,z,h,j,k;
4   FUNCTION INT main()
5         BEGIN
6         STRING str:="Hello";
7         a := 1; b:=2;
8         IF (a = b)
9             j := 0;
10            WHILE (j <= 10)
11                     a := j;
12                     j := j+1;
13            ENDWHILE
14        ENDIF
15        RETURN a+b;
16        END
17 END
```

`scanner.l` file

A global variable to keep track of line numbers

`yylval` is the name of the data object passed from lexer to parser. `yylval` is the variable name. This variable is of type YYSTYPE, which is defined as a union in parser.

`intliteraldata` is the name (of 'field') in the data object's fields defined

type of the field is `std::pair<int, int>`

```
%{
#include<iostream>
#include<cstring>
#include "microParser.tab.hpp"
int lineCount=1;
%}

%%
[0-9]+              { yylval.intliteraldata=new std::pair<int, int>(); (yylval.intliteraldata)->first=atoi(yytext)
;(yylval.intliteraldata)->second=lineCount;return INTLITERAL;}
\"[^\"]*\"          { yylval.stringliteraldata=new std::pair<std::string, int>; (yylval.stringliteraldata)->first=
std::string(yytext);(yylval.stringliteraldata)->second=lineCount;return STRINGLITERAL;}
\n                  {lineCount++;}
```

type of the field is `std::pair<std::string, int>`

increment counter tracking line number when 'newline' symbol is seen in input file

`microParser.ypp` file

`iostream` is needed for `std::pair`
Whatever you put in between %{ and %} is copied to microParser.tab.cpp file but not microParser.tab.hpp file.

This is the type of the data object passed from lexer to scanner. This object is a union and has fields `intliteraldata` and `stringliteraldata`. This union can contain only basic data types such as `int`, `float`, `char`, and pointers.

As the details of this type is required by the scanner and as this type contains `std::string` and `std::pair`, `scanner.l` needs to include corresponding headers. This inclusion must be done before `#include"microParser.tab.hpp"` because the `.hpp` file does not include these headers as mentioned above. Also note that all the fields inside `%union` are pointers types.

```
%{
#include<iostream>
int yylex();
void yyerror(char const* errmsg);
%}

%union{
std::pair<int, int>* intliteraldata;
std::pair<std::string, int>* stringliteraldata;
}

%token <intliteraldata> INTLITERAL
%token STRINGLITERAL
%type <stringliteraldata> STRINGLITERAL
```

Note two ways of associating fields of the data object with tokens: 1) `%token <intliteral> INTLITERAL`. In this case you don't have to separately define `%token INTLITERAL` 2) `%token STRINGLITERAL` followed by `%type<stringliteraldata> STRINGLITERAL`. In this case, we mean explicitly the type of `stringliteral` is the type of the semantic record of STRINGLITERAL token.

```
%%
program: PROGRAM id _BEGIN pgm_body END {printf("Accepted\n");return 0;};

    ..other CFG rules go here

str: STRINGLITERAL {printf("STRINGLITERAL val:%s at line:%d\n",(($1)->first).c_str(), ($1)->second);delete $1;};
primary: LPAREN expr RPAREN {}
       | id {}
       | INTLITERAL {printf("INTLITERAL val:%d at line:%d\n",($1)->first, ($1)->second);delete $1;}
       | FLOATLITERAL {};
```

- ($1) is the reference to data object of STRINGLITERAL. Whenever the category of the token matched in scanner is a STRINGLITERAL, the scanner creates an object of type as that of stringliteraldata, initializes it with appropriate values and sends it to parser. The return statement in scanner only returns the token category. The token value is set by the scanner using `yylval` object. The parser refers to this object using $1 in this case in the semantic action using ($1).
- c_str() is needed to convert std::string to C-style strings (char *). If you use cout to print you don't need this.
- `first` and `second` are the names by which you access the fields of a `std::pair` object
- delete the object created if no longer required.